

Révision**Exercice 1 :**

Dans cet exercice, on travaille dans un environnement Linux. On considère l'arborescence de fichiers de la figure 1.

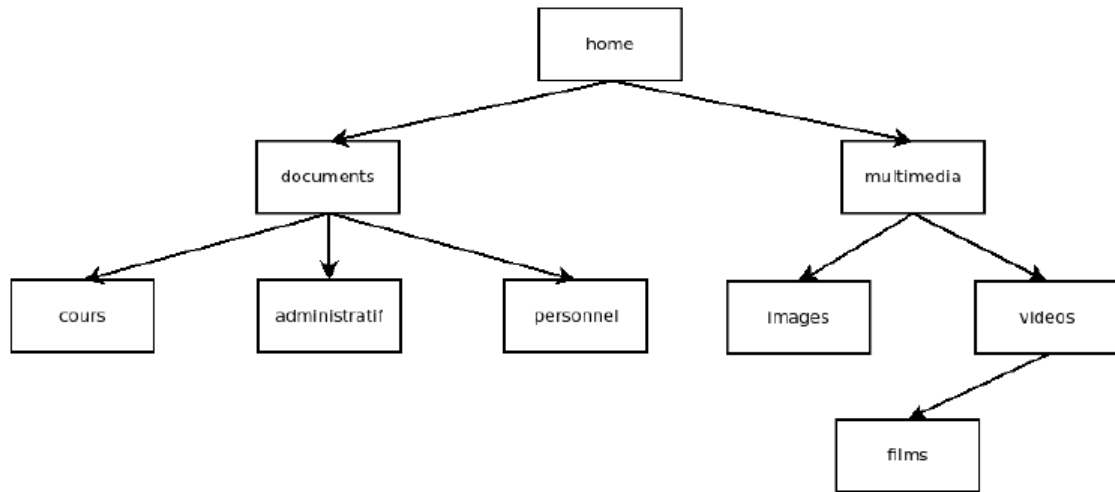


FIGURE 1 – Arborescence de fichiers

Partie A

1. Le répertoire courant est home. Donner une commande permettant de connaître le contenu du dossier documents.
2. On suppose que l'on se trouve dans le dossier cours et que l'on exécute la commande `mv ../../multimedia/home/documents`. Indiquer la modification que cela apporte dans l'arborescence de la figure 1.

On considère le code suivant :

```
class Arbre:
    def __init__(self, nom, g, d):
        self.nom = nom
        self.gauche = g
        self.droit = d
    def est_vide(self):
        return self.gauche is None and self.droit is None

    def parcours(self):
        print(self.nom)
        if self.gauche != None:
            self.gauche.parcours()
        if self.droit != None:
            self.droit.parcours()
```

3. Donner une raison qui justifie que le code précédent ne permet pas de modéliser l'arborescence de fichiers de la figure 1.
4. Donner le nom du parcours réalisé par le code précédent.
5. Donner la liste des dossiers dans l'ordre d'un parcours en largeur de l'arborescence. On ne demande pas d'écrire ce parcours en Python.

Partie B

Pour pouvoir modéliser l'arborescence de fichiers de la figure 1, on propose l'implémentation suivante. L'attribut `files` est une variable de type `list` contenant tous les dossiers fils. Cette liste est vide dans le cas où le dossier est vide.

```
class Dossier:
    def __init__(self, nom, liste):
        self.nom = nom
        self.files = liste # liste d'objets de la classe Dossier
```

6. Écrire le code Python d'une méthode `est_vide` qui renvoie `True` lorsque le dossier est vide et `False` sinon.
7. Écrire le code Python permettant d'instancier une variable `var_multimedia` de la classe `Dossier` représentant le dossier multimedia de la figure 1. Attention : cela nécessite d'instancier tous les nœuds du sous-arbre de racine multimedia.
8. Recopier et compléter sur votre copie le code Python de la méthode `parcours` suivante qui affiche les noms de tous les descendants d'un dossier en utilisant l'ordre préfixe.

```
def parcours(self):
    print(...)
    for f in ...:
        ...
```

9. Justifier que cette méthode `parcours` termine toujours sur une arborescence de fichiers.
10. Proposer une modification de la méthode `parcours` pour que celle-ci effectue plutôt un parcours suffixe (ou postfixe).
11. Expliquer la différence de comportement entre un appel à la méthode `parcours` de la classe `Dossier` et une exécution de la commande UNIX `ls`.
On considère la variable `var_videos` de type `Dossier` représentant le dossier videos de la figure 1. On souhaite que le code Python `var_videos.mkdir("documentaires")` crée un dossier `documentaires` vide dans le dossier `var_videos`.
12. Écrire le code Python de la méthode `mkdir`.
13. Écrire en Python une méthode `contient(self, nom_dossier)` qui renvoie `True` si l'arborescence de racine `self` contient au moins un dossier de nom `nom_dossier` et `False` sinon.
14. Avec l'implémentation de la classe `Dossier` de cette partie, expliquer comment il serait possible de déterminer le dossier parent d'un dossier donné dans une arborescence donnée. On attend ici l'idée principale de l'algorithme décrite en français. On ne demande pas d'implémenter cet algorithme en Python.
15. Proposer une modification dans la méthode `__init__` de la classe `Dossier` qui permettrait de répondre à la question précédente beaucoup plus efficacement et expliquer votre choix.

Exercice 2 :

Amélie souhaite organiser sa collection de CD. Elle a commencé par enregistrer toutes les informations sur un fichier CSV mais elle trouve que la recherche d'informations est longue et fastidieuse. Elle repense à son cours sur les bases de données et elle se dit qu'elle doit pouvoir utiliser une base de données relationnelle pour organiser sa collection.

Partie A

Dans cette partie on utilise une seule table. Voici un extrait de la table chanson.

Chanson			
id	titre	album	groupe
1	Sunburn	Showbiz	Muse
2	Muscle Museum	Showbiz	Muse
3	Showbiz	Showbiz	Muse
4	New Born	Origin of Symmetry	Muse
5	Sing for Absolution	Absolution	Muse
6	Hysteria	Absolution	Muse
7	Welcome too the Jungle	Appetite for Destruction	Guns N' Roses
8	Muscle Museum	Hullabaloo	Muse
9	Showbiz	Hullabaloo	Muse

1. L'attribut titre peut-il être une clé primaire pour la table chanson? Justifier.
2. Donner le résultat de la requête suivante :

```
SELECT titre, album
FROM Chanson
WHERE groupe = 'Guns N'Roses';
```

3. Écrire une requête SQL permettant d'obtenir tous les titres des chansons de l'album Showbiz dans l'ordre croissant.
4. Écrire une requête SQL permettant d'ajouter la chanson dont le titre est Megalomania de l'album Hullabaloo du groupe Muse.
Amélie a remarqué une faute de frappe dans la chanson Welcome too the Jungle qui s'écrit normalement Welcome to the Jungle.
5. Écrire une requête SQL permettant de corriger cette erreur.

Partie B

Dans cette partie on utilise trois tables. Voici des extraits des trois tables Chanson, Album et Groupe.

Chanson		
id	titre	id_album
1	Sunburn	1
2	Muscle Museum	1
3	Showbiz	1
4	New Born	2
5	Sing for Absolution	4
6	Hysteria	4
7	Welcome to the Jungle	5
8	Muscle Museum	3
9	Showbiz	3

Album			
id	titre	année	id_groupe
1	Showbiz	1999	1
2	Origin of Symmetry	2001	1
3	Hullabaloo	2002	1
4	Absolution	2003	1
5	Appetite for Destruction	1987	2

Groupe	
id	nom
1	Muse
2	Guns N' Roses

6. Expliquer l'intérêt d'utiliser trois tables Chanson, Album et Groupe au lieu de regrouper toutes les informations dans une seule table.

7. Expliquer le rôle de l'attribut `id_album` de la table `chanson`.
8. Proposer alors un schéma relationnel pour cette version de la base de données. On pensera à bien spécifier les clés primaires en les soulignant et les clés étrangères en les faisant précéder par le symbole `#`.
9. Écrire une requête SQL permettant d'obtenir tous les noms des albums contenant la chanson Showbiz.
10. Écrire une requête SQL permettant d'obtenir tous les titres avec le nom de l'album des chansons du groupe Muse.
11. Décrire par une phrase ce qu'effectue la requête SQL suivante :

```
SELECT COUNT(*) AS tot
FROM Album AS a
JOIN Groupe AS g ON a.id_groupe = g.id
WHERE g.nom = 'Muse';
```

Partie C

Dans cette partie, on utilise Python. Amélie a remarqué que son professeur ne parle jamais d'ordre alphabétique mais d'ordre lexicographique lorsqu'il fait une requête avec `ORDER BY`.

Elle a compris qu'il s'agissait de l'ordre du dictionnaire mais elle se demande comment elle pourrait elle-même écrire une fonction `ordre_lex` (`mot1`, `mot2`) de comparaison entre deux chaînes de caractères en utilisant l'ordre lexicographique. La fonction

`ordre_lex` (`mot1`, `mot2`) prend en arguments deux chaînes de caractères et renvoie un booléen. Une rapide recherche lui permet de trouver le résultat suivant :

Lorsque l'on compare deux chaînes de caractères suivant l'ordre lexicographique, on commence par comparer les deux premiers caractères de chacune des deux chaînes, puis en cas d'égalité on s'intéresse au second, et ainsi de suite. Le classement est donc le même que celui d'un dictionnaire. Si lors de ce procédé on dépasse la longueur d'une seule des deux chaînes, elle est considérée plus petite que l'autre. Lorsqu'on dépasse la longueur des deux chaînes au même moment, elles sont nécessairement égales

Amélie commence par écrire quelques assertions que sa fonction devra vérifier.

12. Compléter les assertions suivantes :

```
assert ordre_lex("", "a") == True
assert ordre_lex("b", "a") == ...
assert ordre_lex("aaa", "aaba") == ...
```

On suppose que les chaînes de caractères `mot1` et `mot2` ne sont composées que des lettres de l'alphabet, en minuscule, et la comparaison entre deux lettres peut se faire avec les opérateurs classiques `==` et `<`.

13. Recopier et compléter la fonction récursive `ordre_lex` ci-dessous qui prend pour paramètre deux chaînes de caractères `mot1` et `mot2` et qui renvoie `True` si `mot1` précède `mot2` dans l'ordre lexicographique.

```
def ordre_lex(mot1, mot2):
    if mot1 == "":
        return True
    elif mot2 == "":
        return False
    else:
        c1 = mot1[0]
        c2 = mot2[0]
        if c1 < c2:
            return ...
        elif c1 > c2:
            return ...
        else:
            return ...
```

14. Proposer une version itérative de la fonction `ordre_lex`.

On considère le réseau représenté ci-dessous :



Partie B : Protocoles de routage

On donne ci-dessous des extraits des tables de routage des routeurs :

Routeur	Réseau destinataire	Passerelle	Interface
A	L2	53.10.10.2	53.10.10.1
B	L2	193.55.24.6	193.55.24.5
C	L2	193.55.24.10	193.55.24.9
D	L2	Connecté	172.16.255.254
E	L2	42.4.10.14	42.4.10.13
F	L2	42.4.10.10	42.4.10.9
G	L2	53.10.10.13	53.10.10.14
H	L2	53.10.10.6	53.10.10.5

3. À l'aide des extraits des tables de routage ci-dessus, donner un chemin (c'est-à-dire nommer les routeurs traversés) suivi par un message envoyé du réseau L1 vers le réseau L2.

La liaison entre les routeurs H et D est rompue :

4. Sachant que le protocole de routage RIP est utilisé (distance en nombre de sauts), donner les nouveaux chemins que pourra suivre un message allant de L1 vers L2.
5. Choisir un des chemins de la question précédente. Donner les routeurs dont la règle de routage à destination de L2 est obligatoirement modifiée. Après avoir examiné tous les routeurs, écrire sur votre copie les règles de routage modifiées en conséquence.

La liaison entre les routeurs H et D est rétablie.

Pour tenir compte du débit des liaisons, on décide d'utiliser le protocole OSPF (distance liée au coût des liaisons) pour effectuer le routage. Le coût d'une liaison est donné ici par la formule :

$$\text{coût} = \frac{10^9}{\text{BP}}$$

où BP est la bande passante de la connexion en bit par seconde. Les valeurs des bandes passantes de chaque liaison entre les routeurs sont données ci-dessous :

Liaison	Bande passante	Liaison	Bande passante
A-B	1 Gbit/s	D-H	100Mbit/s
A-H	1 Gbit/s	D-E	10Gbit/s
A-G	1 Gbit/s	E-F	10Gbit/s
B-C	1 Gbit/s	F-H	1Gbit/s
C-H	100Mbit/s	F-G	10Gbit/s
C-D	1 Gbit/s	G-H	1Gbit/s

6. Calculer le coût des liaisons pour les 3 valeurs de bande passante qui apparaissent dans le tableau ci-dessus.
7. Déterminer alors le chemin que suivra un message allant de L1 vers L2 et donner son coût.
8. La liaison entre les routeurs G et F est rompue. Déterminer le nouveau chemin suivi par un message allant de L1 vers L2 et donner son coût.